

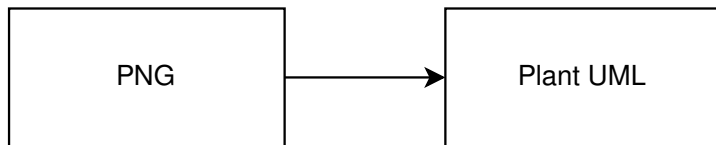
Automatic Extraction of UML Class Diagram Structure from Images

Viktor Ovchinnikov

April 28, 2026

Problem & Goal

- Input: **UML class diagram images** (dataset: **655** images)
- Output: **normalized PlantUML** per image
- Two evaluation tracks:
 - **Scale**: generate PlantUML for full dataset + syntactic validation
 - **Quality**: gold subset + structure-level metrics (classes/edges/attrs/methods)



- Cloud multimodal:
 - **GPT-5.2**
 - **Gemini 2.5 Flash**
- Local baseline (negative result):
 - **LLaVA v1.5 (7B)** (quality/format instability)

Pipeline (high-level)

- ① UML image
- ② Vision model inference → **PlantUML-only** output
- ③ Post-processing:
 - enforce @startuml...@enduml
 - parse to normalized structure graph
 - compare with GT on gold subset
- ④ Save <id>.puml + evaluation logs

Evaluation: Micro-averaged F1 (Gold subset)

Gold subset: **14** diagrams (same IDs for both models)

- Elements: **classes, edges, attributes, methods**
- Metrics: precision / recall / **micro F1** aggregated over all subset diagrams

Micro-averaged structure extraction metrics on the gold subset (14 diagrams).

Element	Model	Precision	Recall	F1
Classes	GPT-5.2	0.966	0.966	0.966
	Gemini 2.5 Flash	0.973	0.979	0.976
Edges	GPT-5.2	0.809	0.620	0.702
	Gemini 2.5 Flash	0.702	0.707	0.704
Attributes	GPT-5.2	0.364	0.194	0.253
	Gemini 2.5 Flash	0.472	0.606	0.531
Methods	GPT-5.2	0.526	0.647	0.580
	Gemini 2.5 Flash	0.624	0.554	0.587

Key takeaways (from current results)

- **Classes:** both models are nearly perfect and close
- **Edges:** GPT is more accurate but misses more often; Gemini more often adds, but adds unnecessary information in some cases
- **Attributes:** Gemini is significantly stronger (according to the current gold subset)
- **Methods:** similar, but there is sensitivity to parsing (attr/method boundary)

Prompt used (verbatim)

You are an assistant that reads UML CLASS DIAGRAMS from images and rewrites them as normalized PlantUML code.

Your task:

Given the image of a UML class diagram, output the corresponding PlantUML code that describes the classes and relationships you can clearly see.

Rules:

1. Output ONLY PlantUML code. Do NOT output any explanations, comments or Markdown code fences.
2. Always start with:

```
@startuml
...classes and relationships...
@enduml
```
3. For each class, use the syntax:

```
class ClassName {
+attribute: Type
+operation(param: Type): ReturnType
}
```

If you cannot read attributes or operations, you may omit the body and use just:

```
class ClassName
```
4. Use EXACT class names, attribute names and operation names from the diagram.
If you cannot read a name, SKIP that element instead of guessing.
5. For inheritance, use:

```
ParentClass <|-- ChildClass
```
6. For associations, use:

```
ClassA -- ClassB
```

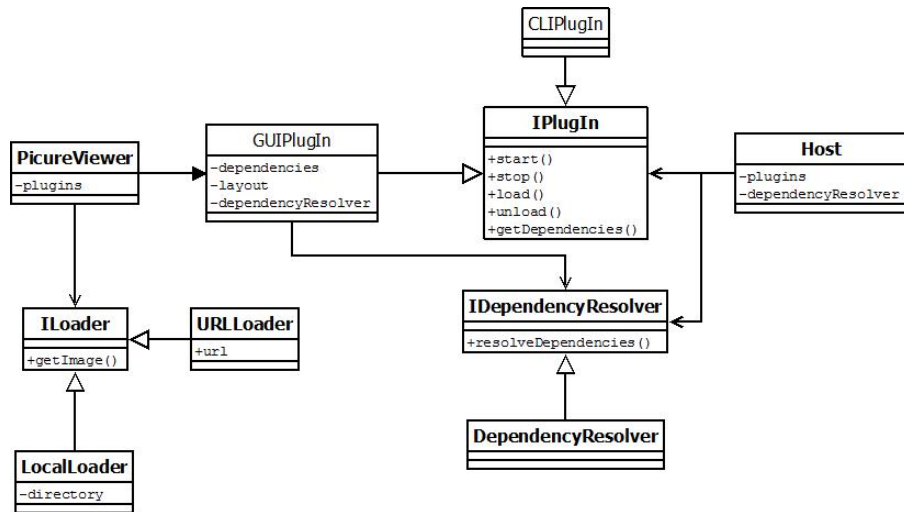
Optionally with multiplicities in quotes, for example:

```
ClassA "1" -- "*" ClassB
```
7. For composition and aggregation, use:

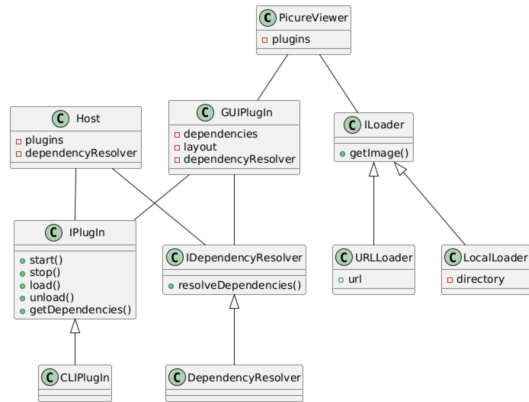
```
ClassA *-- ClassB      (composition)
ClassA o-- ClassB      (aggregation)
```
8. Do NOT invent classes, attributes, operations or relationships that are not present in the image.
9. Do NOT add comments inside the PlantUML code.

If the diagram is very large, include only the main classes and relationships that are clearly readable.

Case A (ID 231): Ground Truth

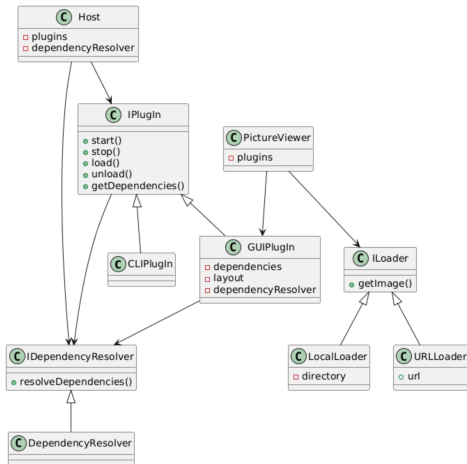


Case A (ID 231): GPT-5.2



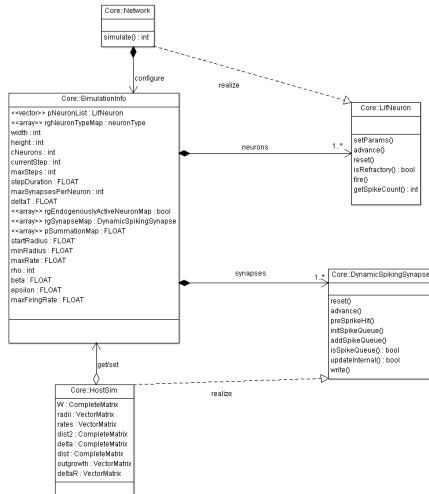
Rendered PlantUML comparison; predicts pictureviewer instead of pictureviewer.

Case A (ID 231): Gemini 2.5 Flash

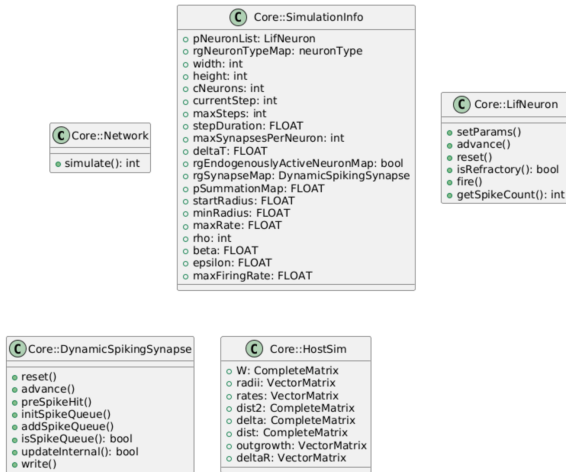


Rendered PlantUML comparison; includes a non-existing edge in this case.

Case B (ID 177): Ground Truth

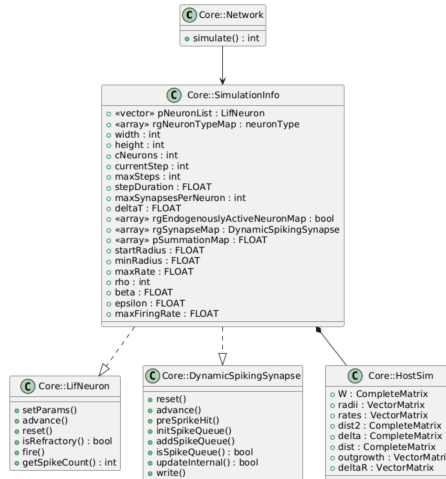


Case B (ID 177): GPT-5.2



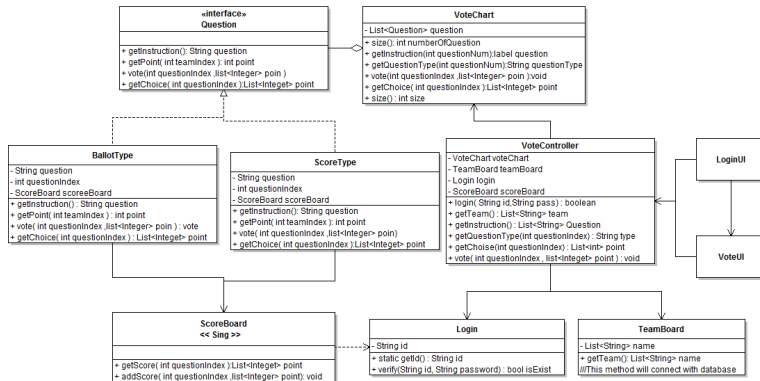
Rendered PlantUML comparison; attributes are weakly recovered and edges are missed entirely.

Case B (ID 177): Gemini 2.5 Flash

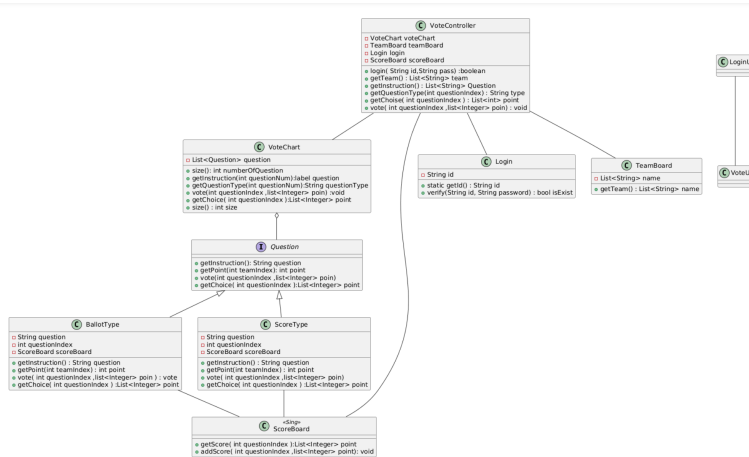


Rendered PlantUML comparison; attributes are weakly recovered with attribute/method confusion.

Case C (ID 305): Ground Truth

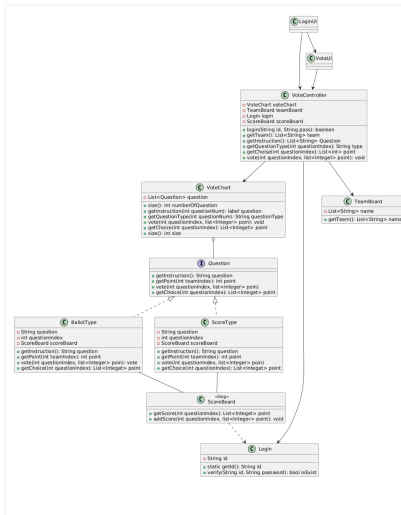


Case C (ID 305): GPT-5.2



Rendered PlantUML comparison; extracts fewer class connections,
with formatting differences such as name:type vs type name.

Case C (ID 305): Gemini 2.5 Flash



Rendered PlantUML comparison; captures more class connections in this diagram.

- **Identifier normalization:** whitespace/punctuation/typos sensitivity
- **Attribute vs method parsing:** better heuristics for signatures and formatting
- **Edges:** reduce spurious edges + recover missed ones on dense diagrams
- Expand gold subset for more stable conclusions